

Sesión 01. Ejercicios Propuestos

Introducción a la Programación. Versión curso.2027

Tabla de contenidos

Objetivos

1. PARTE I: Ejercicios en Java
 - 1.1. Ejercicio Circunferencia
 - 1.2. Ejercicio Distancia
 - 1.3. Ejercicio EnteroAleatorio
 - 1.4. Ejercicio CalculadoraPPI
 - 1.5. Ejercicio Relacion1Ej03
 - 1.6. Ejercicio Relacion1EjXX
 - 1.7. Ejercicio DescomposicionMonedas
 2. PARTE II: Transición a Python
 - 2.1. Diferencias Estructurales Básicas
 - 2.2. Transición: Ejercicio Circunferencia
 - 2.3. Transición: Ejercicio Distancia
 - 2.4. Transición: Ejercicio EnteroAleatorio
 - 2.5. Transición: Ejercicio CalculadoraPPI
 - 2.6. Transición: Ejercicios Relacion1Ej03 y Relacion1EjXX
 - 2.7. Transición: Ejercicio DescomposicionMonedas
-

Objetivos

- Implementar algoritmos sencillos que hagan uso de asignaciones, expresiones y salida de información. Primero se abordarán en **Java** como lenguaje principal y, posteriormente, se adaptarán las soluciones a **Python**.
- Utilizar métodos y funciones de las librerías matemáticas (`Math` en Java y módulo `math` en Python).
- Desarrollar programas que realicen cálculos geométricos utilizando fórmulas matemáticas.
- Practicar la generación de números aleatorios y la manipulación de datos para resolver problemas del mundo real (`Math.random()` en Java y módulo `random` en Python).
- Promover buenas prácticas en la nombrado de clases y variables (convención **CamelCase** en Java y **snake_case** en Python), así como en la documentación del código a través de comentarios.

Novedad del curso 2026-2027 y esquema de paquetes:

En este curso, la práctica se divide en dos partes. Primero deberá resolver todos los problemas en **Java**. Una vez resueltos, los reescribirá en **Python** prestando atención a las diferencias sintácticas.

- **Para la Parte I (Java):** Cree el paquete `org.ip2027.sesion01` dentro del directorio `src/main/java`. Cada programa se creará como una clase con el nombre indicado en **negrita** (`NombrePrograma.java`).
- **Para la Parte II (Python):** Utilice la carpeta de paquetes equivalente `org/ip2027/sesion01` dentro de `src/main/python`. Cada programa se creará como un archivo `.py` con el nombre equivalente en minúsculas y guiones bajos (`nombre_programa.py`).



```
▼ 📁 src/main/java
  > 📁 org.ip2027.sesion01
▼ 📁 src/main/python
  > 📁 org.ip2027.sesion01
```

1. PARTE I: Ejercicios en Java

A continuación se plantean una serie de problemas clásicos de programación. Céntrese en resolver la lógica del problema utilizando la sintaxis de Java.

1.1. Ejercicio Circunferencia

Dado el radio de una circunferencia, desarrolle el programa en Java (`Circunferencia.java`) que permita calcular la longitud de dicha circunferencia y el área del círculo que hay en su interior. Para ello, debe tener en cuenta que el radio es una variable real, que el valor de la constante π está disponible en la librería matemática de Java (`Math.PI`) y que hay que utilizar las siguientes fórmulas (siendo `r` el radio):

$$\text{Longitud} = 2 \cdot \pi \cdot r$$

$$\text{Area} = \pi \cdot r^2$$

Muestre por pantalla los mensajes exactamente como se indica a continuación. Para ello recuerde utilizar la función `System.out.printf` con el formato `%.3f`.

Ejemplo de ejecución:

```
Radio = 4.57
```

```
Diametro de la circunferencia = 9.14
```

```
Perimetro de la circunferencia = 28,714
```

```
Area de la circunferencia = 65,612
```

Además, busque en Wikipedia las fórmulas del volumen y área de una esfera. Añada al programa anterior el cálculo del volumen y área de la esfera del mismo radio. Muestre por pantalla los nuevos resultados de volumen y área con mensajes similares a cómo ha realizado en la primera parte del ejercicio.

1.2. Ejercicio Distancia

Desarrolle un programa en Java (`Distancia.java`) que muestre la distancia de un punto (x, y) al origen $(0, 0)$, donde x e y son enteros. Compruebe con distintas ejecuciones las distancias que se indican a continuación:

- Distancia de $(3, 4)$ a $(0, 0) = 5.0$
- Distancia de $(5, 12)$ a $(0, 0) = 13.0$
- Distancia de $(2, 1)$ a $(0, 0) = 2.23606797749979$.

Ejemplo de ejecución:

```
La distancia del punto (2, 1) al punto (0, 0) es 2.23606797749979
```



Para usar las funciones matemáticas que necesite en Java, puede utilizar los métodos de la clase `Math` (del paquete `java.lang`, no requiere `import`): - `Math.sqrt(x)`: raíz cuadrada de `x`. - `Math.pow(x, y)`: `x` elevado a `y`.

1.3. Ejercicio EnteroAleatorio

Desarrolle un programa en Java (`EnteroAleatorio.java`) que muestre un entero pseudo-aleatorio entre -15 y 15. Pruebe distintas ejecuciones utilizando el método `Math.random()` .

Debe tener en cuenta que dado un número aleatorio generado entre 0.0 y 1.0 ($0.0 \leq \text{aleatorio} < 1.0$), para conseguir un número entero entre M y N con $M < N$ y ambos incluidos, se debe usar la siguiente fórmula:

$$\text{Valor Entero} = \lfloor \text{aleatorio} \cdot (N - M + 1) \rfloor + M$$

Ejemplo de ejecución:

```
Vamos a generar un entero aleatorio entre -15 y 15
```

```
El entero generado aleatoriamente es: -14
```

1.4. Ejercicio CalculadoraPPI

Escriba un programa en Java (`CalculadoraPPI.java`) que calcule y muestre la densidad PPI para cada uno de estos dispositivos, utilizando los valores codificados directamente en el código fuente.

La densidad PPI indica cuántos píxeles hay en cada pulgada de pantalla, y se puede calcular con la siguiente fórmula:

$$PPI = \frac{\sqrt{(ancho_resolucion^2 + alto_resolucion^2)}}{tamaño_diagonal}$$

- `ancho_resolucion` es el número de píxeles a lo ancho de la pantalla.
- `alto_resolucion` es el número de píxeles a lo alto de la pantalla.
- `tamaño_diagonal` es el tamaño de la pantalla en pulgadas.

A continuación, se dan las resoluciones y tamaños de pantalla para varios dispositivos:

1. Monitor Full HD de 27 pulgadas:

- Resolución: 1920x1080 píxeles.
- Tamaño de la pantalla: 27 pulgadas.

2. Monitor 4K de 32 pulgadas:

- Resolución: 3840x2160 píxeles.
- Tamaño de la pantalla: 32 pulgadas.

3. Dispositivo móvil de 6.5 pulgadas:

- Resolución: 2340x1080 píxeles.
- Tamaño de la pantalla: 6.5 pulgadas.



Para un ordenador de escritorio, un PPI adecuado generalmente varía entre 90 y 110. En pantallas grandes, como monitores de 27 o 32 pulgadas, una densidad de píxeles en este rango es suficiente para obtener una calidad de imagen clara sin que los píxeles individuales sean perceptibles a una distancia de trabajo típica (50-80 cm). Por ejemplo, un monitor Full HD de 27 pulgadas tiene alrededor de 81 PPI, que es aceptable, pero en un monitor 4K de 32 pulgadas, el PPI aumenta a 137, proporcionando una imagen más nítida.

En dispositivos móviles, dado que se usan a distancias mucho más cercanas (20-30 cm), un PPI mayor es necesario para evitar que los píxeles sean visibles. Un PPI de al menos 300 se considera adecuado para móviles de alta gama, como los smartphones modernos, donde se busca una mayor definición y claridad en pantallas más pequeñas.

Para obtener más información consulta:

- [Resolución en el monitor](https://hardzone.es/tutoriales/rendimiento/monitor-resolucion-calidad-imagen/) (<https://hardzone.es/tutoriales/rendimiento/monitor-resolucion-calidad-imagen/>).
- [Calculadora de PPI](https://onlinerulers.com/es/pixel-density/) (<https://onlinerulers.com/es/pixel-density/>)

Ejemplo de salida:

La densidad de PPI del monitor Full HD de 27 pulgadas es: 81.59 PPI.
La densidad de PPI del monitor 4K de 32 pulgadas es: 137.68 PPI.
La densidad de PPI del dispositivo móvil de 6.5 pulgadas es: 396.49 PPI.

1.5. Ejercicio Relacion1Ej03

Desarrolle un programa en Java (`Relacion1Ej03.java`) que resuelva el ejercicio 3 de la Relación 1 de Física:

Un cilindro de 10 cm de altura y radio de la base 3 cm tiene una densidad homogénea de carga volumétrica de $1 \mu\text{C} \cdot \text{m}^{-3}$. Calcular la carga total que almacena

Muestre por pantalla: el enunciado del ejercicio, los datos y la solución.

Ejemplo de ejecución (orientativo):

```
Solución al ejercicio 3 de la relación 1: Relacion1Ej03
=====
ENUNCIADO:
3) Un cilindro de 10 cm de altura y radio de la base 3 cm tiene una densidad homogénea
de carga volumétrica de 1  $\mu\text{C}/\text{m}^3$ . Calcular la carga total que almacena.
=====
DATOS:
Altura = 0.1 cm
Radio = 0.03 cm
Densidad de carga = 1.0E-6
=====
SOLUCIÓN:
La carga total almacenada es: 2.8274333882308136E-10 C
=====
```

1.6. Ejercicio Relacion1EjXX

Desarrolle un programa en Java (`Relacion1EjXX.java`) que resuelva otro ejercicio a elegir de la Relación 1 de Física. En el nombre de la clase Java sustituya `XX` por el número de ejercicio de la relación 1 de Física que haya elegido.

Use el mismo esquema del ejercicio anterior para mostrar los resultados por pantalla.

1.7. Ejercicio DescomposicionMonedas

Desarrolle un programa en Java (`DescomposicionMonedas.java`) que, a partir de una cantidad entera fija expresada en céntimos de euro (por ejemplo, 487 céntimos), determine el desglose óptimo en el menor número posible de monedas de curso legal: 2 euros, 1 euro, 50 céntimos, 20 céntimos, 10 céntimos, 5 céntimos, 2 céntimos y 1 céntimo.

Para ello, resuelva el cálculo de forma estrictamente secuencial empleando divisiones enteras (`/`) para calcular el número de monedas de cada denominación y el operador resto o módulo (`%`) para obtener los céntimos restantes que pasan al siguiente desglose.

Ejemplo de ejecución:

```
Cantidad inicial: 487 céntimos
```

```
Monedas de 2 euros: 2  
Monedas de 1 euro: 0  
Monedas de 50 céntimos: 1  
Monedas de 20 céntimos: 1  
Monedas de 10 céntimos: 1  
Monedas de 5 céntimos: 1  
Monedas de 2 céntimos: 1  
Monedas de 1 céntimo: 0
```

2. PARTE II: Transición a Python

Una vez resueltos todos los problemas algorítmicos en Java, el objetivo de esta segunda parte es recodificar las soluciones en **Python**. Aproveche que ya tiene la lógica diseñada y céntrese en las particularidades de la sintaxis de este lenguaje.

Cree los archivos Python en el paquete `org/ip2027/sesion01` de la carpeta `src/main/python` usando el entorno Eclipse con PyDev que ya tiene configurado. A continuación, se ofrecen unos *tips* (consejos) para facilitar la transición en cada ejercicio.

2.1. Diferencias Estructurales Básicas

Antes de comenzar la traducción a Python, tenga presente las siguientes reglas esenciales de sintaxis y estructura:



Estructura del script y función `main`

- **Función `main`**: En Python no es estrictamente obligatorio envolver el código en una clase (`public class ...`) ni definir un método `main`, ya que cualquier archivo `.py` puede ejecutarse directamente instrucción a instrucción. No obstante, **en esta asignatura hemos optado por estructurar todos los programas de Python dentro de una función `main()`** y realizar su llamada al final del archivo. Esto ayuda a mantener una estructura limpia, modular y coherente con el paradigma aprendido en Java.
- **Bloques delimitados por indentación**: En Java los bloques de código se delimitan con llaves (`{ ... }`). En Python no existen las llaves para bloques: **los bloques de código se definen exclusivamente mediante la indentación** (habitualmente cuatro espacios en blanco después de los dos puntos `:`). Una indentación incorrecta provoca un error de sintaxis (`IndentationError`).
- **Fin de instrucción sin punto y coma**: No escriba punto y coma (`;`) al final de las sentencias; el salto de línea delimita cada instrucción.
- **Comentarios**: Los comentarios de una línea comienzan con la almohadilla (`#`), sustituyendo a la doble barra (`//`) de Java.

Table 1. Comparativa Rápida de Sintaxis

Java	Python
<pre>package org.ip2027.sesion01; public class Circunferencia { public static void main(String[] args) { // Comentario de una línea double radio = 4.57; System.out.printf("R = %.2f\n", radio); } }</pre>	<pre># Comentario de una línea def main(): radio = 4.57 print(f"R = {radio:.2f}") if __name__ == "__main__": main()</pre>

2.2. Transición: Ejercicio Circunferencia

Desarrolle el equivalente en el archivo `circunferencia.py`.

- **Tipado dinámico:**

En Python no se declaran los tipos de las variables; basta con asignar el valor directamente:

```
radio = 4.57
```

PYTHON

- **Librería matemática y módulo `math`:**

En Java la clase `Math` se encuentra en el paquete base `java.lang`, el cual es importado automáticamente por el compilador en todas las clases, por lo que nunca necesita escribir `import java.lang.Math;`. En cambio, en Python la biblioteca matemática no está cargada de forma implícita; es imprescindible importar el módulo `math` explícitamente al comienzo del archivo:

```
import math

longitud = 2 * math.pi * radio
```

PYTHON

Salida formateada con f-strings

En Python, la forma idiomática, moderna y más legible para dar formato a textos y variables son las **f-strings** (cadenas literales formateadas precedidas por la letra **f**).

Puede consultar una guía detallada y ejemplos de su funcionamiento en: [Guía completa de f-strings en Python](https://realpython.com/python-f-strings/) (<https://realpython.com/python-f-strings/>).

Para fijar la precisión a 3 decimales (análogo a `%.3f` en `System.out.printf`):

```
print(f"Perimetro de la circunferencia = {perimetro:.3f}")
print(f"Area de la circunferencia = {area:.3f}")
```

PYTHON



2.3. Transición: Ejercicio Distancia

Desarrolle el equivalente en el archivo `distancia.py`.

- **Raíz cuadrada y potencias con `math`:**

```
import math

distancia = math.sqrt(math.pow(x, 2) + math.pow(y, 2))
```

PYTHON



Operador de potencia nativo

Python dispone del operador aritmético nativo `**` para calcular potencias sin necesidad de llamar a `math.pow()`. La expresión anterior se simplifica notablemente:

```
distancia = math.sqrt(x**2 + y**2)
```

PYTHON

2.4. Transición: Ejercicio EnteroAleatorio

Desarrolle el equivalente en el archivo `entero_aleatorio.py`.

- **Generación de aleatorios con `random`:**

En Python, la generación pseudoaleatoria se gestiona mediante el módulo `random`. La función `random.random()` genera un número flotante en el intervalo $[0.0, 1.0)$, equivalente a `Math.random()` en Java.

```
import random

aleatorio = random.random()
```

PYTHON



Para profundizar en las funciones que ofrece este módulo, consulte la documentación oficial: [Módulo random en Python](https://docs.python.org/es/3/library/random.html) (<https://docs.python.org/es/3/library/random.html>).

- **Cálculo del entero:**

Para aplicar la fórmula del enunciado $\lfloor \text{aleatorio} \cdot (N - M + 1) \rfloor + M$ con $M = -15$ y $N = 15$, debe tener cuidado con los números negativos:

1. **Uso riguroso de `math.floor()`:** La función matemática de suelo `math.floor()` redondea siempre hacia el entero inferior más cercano en la recta real (por ejemplo, `math.floor(-14.3)` es `-15`), implementando con total exactitud la fórmula matemática para cualquier intervalo.
2. **Diferencia con `int()`:** La función `int()` no calcula el suelo, sino que **trunca** hacia cero eliminando la parte decimal (`int(-14.3)` devuelve `-14`). Aunque en números positivos `int()` y `math.floor()` coinciden, al trabajar con valores negativos no son equivalentes. Por ello, para este ejercicio se recomienda utilizar explícitamente `math.floor()`.

2.5. Transición: Ejercicio CalculadoraPPI

Desarrolle el equivalente en el archivo `calculadora_ppi.py`.



Convención de nombres en Python: snake_case

A diferencia de Java, donde predomina **CamelCase** (por ejemplo, `anchoResolucion`), la guía de estilo oficial de Python (PEP 8) prescribe el uso de **snake_case** para nombres de variables y funciones (palabras en minúsculas separadas por guion bajo):

- `ancho_resolucion`
- `alto_resolucion`
- `tamano_diagonal`



Comportamiento de la división (/) frente a división entera (//)

En Java, la división entre enteros `int / int` (como `7 / 2`) realiza una división truncada a entero que da `3`, perdiéndose los decimales salvo que se aplique una conversión explícita (**cast**) a `double`.

En Python:

- El operador de división simple `/` siempre realiza una **división real** y devuelve un flotante (`float`), incluso si ambos operandos son enteros:

```
resultado = 7 / 2      # Produce 3.5 (división real)
```

PYTHON

- Si en algún momento se desea la **división entera** (el cociente entero equivalente al operador `/` de enteros en Java), Python dispone del operador específico `//`:

```
resultado_entero = 7 // 2  # Produce 3 (división entera)
```

PYTHON

2.6. Transición: Ejercicios Relacion1Ej03 y Relacion1EjXX

Desarrolle los programas equivalentes en `relacion1_ej03.py` y `relacion1_ejXX.py`.

Cadenas multilínea y formato en bloque

Para imprimir enunciados largos o bloques de salida estructurados, Python permite definir cadenas multilínea encerradas entre tres comillas dobles (`"""..."""`) o triples comillas simples (`'''...'''`).

Vale la pena señalar que en Java moderno también existen los **Text Blocks** utilizando igualmente tres comillas dobles (`"""..."""`), pero en Python su uso está generalizado desde sus orígenes.

Combinadas con una f-string, permiten estructurar toda la presentación en un único `print` :



```
print(f"""=====
ENUNCIADO:
3) Un cilindro de 10 cm de altura y radio de la base 3 cm...
=====
DATOS:
Altura = {altura} cm
Radio = {radio} cm
Densidad de carga = {densidad}
=====
SOLUCIÓN:
La carga total almacenada es: {carga_total} C
=====""")
```

PYTHON

- **Notación científica:**

Se escribe exactamente igual que en Java: `1.0e-6` representa $1.0 \cdot 10^{-6}$.

2.7. Transición: Ejercicio DescomposicionMonedas

Desarrolle el equivalente en el archivo `descomposicion_monedas.py`.

- **División entera (`//`) frente a real (`/`):**

En Java la división entre dos enteros (`int / int`) descarta automáticamente los decimales. En cambio, en Python el operador `/` siempre genera un resultado de tipo `float` (por ejemplo, `487 / 200` devuelve `2.435`).

Para obtener el número exacto de monedas completas sin decimales, recuerde emplear el operador de división entera `//`:

```
# Ejemplo de cálculo de monedas y resto
monedas_200 = cantidad // 200
resto = cantidad % 200
```

PYTHON

“*Patience is the key to joy.*

— Rūmī